Projeto de intervenção — Situação-Problema 3

Nome: _____

RGM: _____

Qual situação-problema você escolheu para criar o seu projeto de intervenção?

[ ] Situação-problema 1

[ ] Situação-problema 2

[X] Situação-problema 3

A Situação-Problema escolhida:

Refatoração e Orientação a Objetos. No cenário apresentado, um estudante implementou o cadastro, a listagem, a consulta de detalhes, a edição, a exclusão e a busca de um Catálogo Simples de Livros/Filmes diretamente em uma Servlet. Essa concentração de responsabilidades produziu um código extenso, difícil de compreender e manter, com repetição de operações de acesso ao banco de dados. A intervenção consiste em reorganizar o catálogo utilizando orientação a objetos e separação de responsabilidades, preservando as funcionalidades exigidas.

O projeto recebeu o nome Acervo e contempla livros, filmes e séries. A solução foi implementada em Java, com Servlets, JSP/JSTL, JDBC e banco relacional H2. A descrição do problema está nas páginas 39–40 do PDF fornecido; as funcionalidades gerais e a documentação solicitada são apresentadas nas páginas 7–9 e 20–28.

Objetivo:

Solucionar a dificuldade de manutenção do catálogo mediante uma arquitetura em camadas, retirando da Servlet as regras de negócio e as instruções SQL. A Servlet passa a interpretar as requisições HTTP e selecionar a resposta; a classe de serviço passa a validar e coordenar as operações; o DAO passa a executar o acesso ao banco; a entidade ItemMidia representa os dados; e as páginas JSP apresentam as informações.

Os objetivos específicos são centralizar o SQL e a conversão de registros em objetos, reduzir a duplicação, facilitar a evolução do código e permitir testes das regras de negócio sem servidor web ou banco real. A aplicação deve manter cadastro, listagem, detalhes, edição, exclusão, busca por título ou autor/diretor e armazenamento permanente dos itens.

Revisão de Conceito:

Foram utilizados os fundamentos de Java e de Programação Orientada a Objetos estudados na disciplina: classes, objetos, atributos, construtores, encapsulamento, interfaces, polimorfismo, coleções e tratamento de exceções. A entidade ItemMidia possui atributos privados e acesso por métodos públicos, enquanto o enum TipoMidia restringe os tipos aceitos a livro, filme e série.

O padrão DAO encapsula as operações de persistência. A interface ItemMidiaDao define o contrato utilizado pelo serviço, e JdbcItemMidiaDao implementa esse contrato com JDBC. Nos testes de serviço, uma implementação em memória substitui o DAO JDBC, demonstrando polimorfismo e injeção de dependências. A herança é reconhecida como um recurso de POO, mas não foi criada uma hierarquia artificial entre mídias: como as três categorias compartilham os mesmos atributos e operações, composição, interface e enum atendem ao escopo com menor complexidade.

Também foram aplicados MVC, responsabilidade única, alta coesão, baixo acoplamento, reutilização de código, CRUD, SQL parametrizado e validação. O tratamento de recursos utiliza try-with-resources; os dados apresentados nas JSPs são escapados para reduzir o risco de XSS. A metodologia de Aprendizagem Baseada em Projetos orienta a integração entre a teoria, a implementação e a reflexão sobre os resultados.

Metodologia:

Foi adotada uma abordagem de desenvolvimento iterativo baseada em projetos, dividida em concepção e planejamento, implementação por funcionalidades, testes e refinamento, documentação e preparação da apresentação. Inicialmente, o problema foi traduzido em requisitos e modelos. Em seguida, foram implementados o modelo de domínio, a persistência, o serviço, o controlador e as telas.

O cenário anterior foi representado por um exemplo didático de Servlet concentrando responsabilidades, pois o material não fornece o código-fonte de uma aplicação legada. A implementação entregue é uma reconstrução organizada da solução solicitada. A equivalência funcional é verificada por testes do CRUD e da busca; não se afirma uma comparação executada contra um sistema anterior inexistente.

Os testes foram distribuídos em três níveis: serviço com DAO em memória, integração JDBC com H2 e integração HTTP com Tomcat real e compilação das JSPs. Foram incluídos casos válidos, dados incorretos, itens inexistentes, entradas com aspas, tentativa de SQL Injection, escape de HTML e proteção de operações POST por token CSRF. O registro dos resultados efetivamente obtidos acompanha a entrega.

Tempo:

O cronograma de referência para execução individual é de 24 horas: 3 horas para análise e requisitos; 3 horas para modelagem; 8 horas para implementação; 4 horas para testes e refinamento; 4 horas para documentação; e 2 horas para revisão e preparação da apresentação.

Esse total é uma estimativa de planejamento, não uma medição das horas trabalhadas pelo estudante. Antes da submissão, deve ser ajustado conforme o tempo efetivamente dedicado à leitura, execução, compreensão, adaptação e revisão do projeto.

Procedimento e material utilizado:

• Ler o material teórico e identificar o problema central da Situação-Problema 3: o acúmulo de responsabilidades e a repetição do acesso ao banco dentro da Servlet.

• Definir os requisitos funcionais do catálogo, os campos de cada item e as regras de validação. Delimitar o uso pessoal e acadêmico, sem login e sem upload de arquivos.

• Produzir os diagramas de casos de uso, classes, entidade-relacionamento e sequência de cadastro. Elaborar o esquema SQL da tabela item_midia.

• Preparar o ambiente com JDK 17 e Maven. Utilizar um editor ou IDE de preferência e navegador web para a interação com o catálogo.

• Implementar ItemMidia e TipoMidia para representar as obras; criar a fábrica de conexões e a inicialização não destrutiva do banco H2.

• Definir ItemMidiaDao e implementar JdbcItemMidiaDao, utilizando PreparedStatement para CRUD e busca. Reutilizar os métodos de preenchimento de parâmetros e mapeamento de resultados.

• Implementar ItemMidiaService para validar campos obrigatórios, limites de tamanho, tipo de mídia e ano, coordenando as operações através do contrato DAO.

• Implementar ItemMidiaServlet para roteamento HTTP, leitura de parâmetros, delegação ao serviço e seleção de JSPs. Adotar Post/Redirect/Get após operações bem-sucedidas.

• Construir as telas de listagem, cadastro, edição, detalhes e erro com JSP/JSTL, HTML, CSS e JavaScript. Aplicar escape de saída e confirmação de exclusão.

• Executar testes de serviço, banco e fluxo HTTP; conferir persistência em arquivo, mensagens de erro e mecanismos de proteção. Registrar o resultado real dos testes.

• Preparar os documentos em Word e PDF, o manual de uso, o guia detalhado do código e as capturas de tela. Revisar os arquivos de entrega e preencher a identificação acadêmica.

Os materiais utilizados são o PDF da disciplina, computador com JDK 17, Maven, Tomcat 10.1, H2, JDBC, Servlets, JSP/JSTL, JUnit 5, editor de código e navegador. Python e bibliotecas de geração de documentos foram utilizados somente na preparação dos documentos, não na execução do catálogo.

Resultado e discussão:

O resultado é uma aplicação web funcional com os recursos exigidos e código organizado por responsabilidade. O acesso SQL está concentrado na implementação DAO, as regras estão no serviço e a Servlet contém o fluxo HTTP. Os dados são persistidos em arquivo H2 e podem ser consultados após o encerramento e a reabertura das conexões. A documentação inclui modelos, instalação, explicação dos arquivos, fluxos, segurança, testes e roteiro de apresentação.

A contribuição da POO é observável na organização em objetos com funções definidas, no encapsulamento e na dependência do serviço em relação a uma interface. O DAO reduz a repetição ao reutilizar o mapeamento dos registros, o preenchimento dos parâmetros e a abertura e o fechamento de recursos. A classe de serviço melhora a testabilidade porque suas regras podem ser executadas com um DAO em memória, sem HTTP ou JDBC.

Os testes automatizados verificam as funcionalidades e os principais cenários de falha; os números e o estado final estão registrados no relatório de verificação da entrega. Não foram medidos ganhos percentuais de produtividade ou desempenho, nem realizadas pesquisas com usuários. A melhoria de manutenção é sustentada pela separação estrutural das responsabilidades e pela possibilidade demonstrada de testar as camadas isoladamente.

As limitações incluem ausência de autenticação, paginação, histórico de alterações e tratamento de edição simultânea com controle de versão. Como continuidade, podem ser adicionados esses recursos, testes com usuários e migração para outro SGBD, mediante adaptação do esquema, do driver e das particularidades SQL. O projeto atende ao escopo acadêmico proposto e evidencia como uma refatoração orientada a objetos organiza o software sem eliminar as funcionalidades originais do enunciado.